

Bitcoin Volatility Prediction

Forecasting BTC's 7-Day Realized Volatility, 168 Hours Ahead, with
GARCH, HAR-RV-X, and EWMA

Mohammed Hejairi · Cohort DSB2

August 2026

Table of Contents

Abstract

This report presents an hourly forecasting pipeline for Bitcoin's 7-day realized volatility — the rolling standard deviation of hourly returns over a 168-hour window — built on hourly BTC/USDT OHLCV data from Binance (as far back as Binance itself has, through 2026-01-01) via CCXT. Volatility, not price direction, is the target: it is a regression problem whose output feeds directly into option pricing, Value-at-Risk, and position-sizing formulas, none of which can act on a directional “up/down” label.

Critically, the model predicts volatility **168 hours (7 days) ahead of the features used to predict it**, not the next hour's value. An earlier version of this project predicted 1 hour ahead and produced a suspiciously high R^2 (near 0.99) for naive persistence and Holt-Winters — not because those models were good, but because a 168-hour rolling target sampled 1 hour apart overlaps 99.4% with its own immediate past, making it trivially easy to “predict” by just repeating the last known value. Moving the horizon out to 168 hours — exactly the window length — removes that overlap entirely and turns this into the genuine, literature-comparable forecasting problem it should be.

Three time-series-native, finance-native models were built, each as a basic version and a tuned improvement: a GARCH-family model (GARCH(1,1) → EGARCH/GJR-GARCH selected by AIC), HAR-RV (→ HAR-RV-X with an ATR regressor and Ridge regularization), and EWMA (RiskMetrics $\lambda=0.94$ → backtested-tuned λ). Two baselines — naive persistence and Holt-Winters exponential smoothing — anchor the comparison. All models are evaluated two ways: on a strict chronological 80/20 historical test split, and on a wide live-test period (2026-01-02 through yesterday) that was never part of the training data at all.

The headline result: after fixing a genuine scale bug in HAR-RV-X's ATR regressor (raw dollar terms vs. fractional volatility, a roughly 45,000x mismatch) and later promoting two further improvements validated by dedicated robustness checks - two additional regressors (EWMA's own tuned sigma and realized skewness) and a genuine log(target) transform, which stack rather than duplicate each other's gains (Sections 7.4.4-7.4.5) - HAR-RV-X is the strongest model on both splits, beating naive persistence by 17.1% RMSE historically and posting the best score on every metric on genuinely unseen live data (R-squared 0.1779, MAPE 27.18%). EWMA, re-tuned over a wider decay-factor grid ($\lambda=0.995$), is a distant second. A wide range of further

modifications - additional technical-indicator regressors, ensembling with EWMA, automated feature selection over the full feature set, alternative realized-volatility estimators, an LSTM given the same indicator set, and a naive $\log_{1p}(x)$ transform that barely changed anything - were tested with the same cross-validation discipline and honestly did not improve on HAR-RV-X, and are reported as negative results rather than omitted; only the regressor and transform combination cleared both a historical and a live-data validation bar, and is the only one promoted into the kept model. GARCH, by contrast, degrades badly at this longer horizon (R-squared -110.68 live) - a structural property of its mean-reverting forecast, not an implementation defect.

1. Introduction and Background

1.1 Motivation

Bitcoin's price swings unpredictably at short (hourly) timescales, and traders, exchanges, and risk desks have no reliable way to anticipate how turbulent the next stretch of trading will be. This leads to mispriced risk, thin margin buffers, and poorly-timed position sizing. An analogy: forecasting price is like predicting exactly where a boat will be tomorrow; forecasting volatility is like predicting how rough the sea will be. The second question is more tractable, and for anyone managing risk it is usually the more useful one.

1.2 Problem Statement and Aim

The aim is to predict Bitcoin's 7-day realized volatility (rolling standard deviation of hourly returns over a 168-hour window), **168 hours ahead of the features used to predict it** — not the next hour's value, which barely differs from the current one and would trivially favor a naive forecast over genuine modeling (Section 8.1 walks through why). Features are lagged price/volume history and technical indicators (RSI, EMA, ATR, Bollinger width) sourced via the CCXT exchange API at hourly resolution. This gives traders and risk desks a continuous, hour-by-hour-updating estimate of next week's volatility usable directly in position-sizing and margin decisions, targeting an RMSE that beats both a naive-persistence baseline and a seasonality-aware Holt-Winters baseline.

Problem type: Regression. Downstream consumers of a volatility number — Black-Scholes σ , Value-at-Risk, position-sizing formulas — need an actual continuous estimate, not a high/low category.

1.3 Why Volatility, Not Price

Use case	How volatility enters
Option pricing	Black-Scholes takes σ as a required input
Value-at-Risk	Loss estimates scale directly with σ
Position sizing	Trade smaller when choppy, larger when calm
Exchange margin	Collateral requirements rise with volatility

1.4 Objectives

- How does BTC’s realized volatility behave over time at hourly resolution — is it clustered (calm stretches punctuated by spikes), and does that clustering persist across many hours (autocorrelation)?
- Does volatility differ systematically by hour-of-day or day-of-week (session-overlap effects)?
- What engineered features (lagged returns, rolling volatility, volume changes, technical indicators) are most predictive of volatility one week out?
- How much does hyperparameter tuning improve each model over its untuned baseline?
- Does GARCH’s asymmetric (leverage-effect) extension actually improve on plain GARCH(1,1) for BTC, the way it does for equities — and does GARCH’s short-horizon design translate to a 7-day-ahead forecast at all?
- Does the best model, trained on historical data, actually generalize to real data collected months after training?

Three models were built, each genuinely time-series-native by construction rather than generic tabular ML fed lag columns: **GARCH-family**, **HAR-RV**, and **EWMA**. Linear Regression, Random Forest, and XGBoost were deliberately excluded — they have no built-in notion of time. An LSTM originally explored in the third model slot was also dropped, in favor of EWMA: all three final models are explicitly finance-native, with a real econometric or risk-management pedigree, rather than a generic sequence learner applied to a financial series.

2. Data

2.1 Source and Collection

Data was sourced via CCXT — a unified exchange API — from Binance’s BTC/USDT hourly OHLCV feed. CCXT was chosen over `yfinance` because `yfinance` is fundamentally an equity-market API: it returns `Dividends/Stock Splits` columns that are always zero for crypto and carry no signal, whereas CCXT pulls exchange-native OHLCV data directly.

The dataset covers a fixed, explicit range. Training data was requested from 2010-01-01 — Bitcoin’s own origin — but **Binance itself did not exist before mid-2017**, so the fetch naturally starts wherever Binance’s real history begins (2017-08-17) rather than at the requested date; no exchange can return trades from before it existed. Training runs through **2026-01-01**. The test period then runs from **2026-01-02 through yesterday** — deliberately wide, not a single week, for a reason specific to a 168-hour-ahead forecast: a prediction can only be checked once 168 real hours have actually elapsed, so a wide test window means many predictions have already had time to be checked rather than the handful a single week right after training would give (Section 9). Data is fetched once and cached; every subsequent cell reads that file rather than hitting the live API again.

2.2 Data Dictionary

Column	Type	Description
Date	datetime	timestamp of the hourly candle (crypto trades 24/7)
Open, High, Low, Close	float	OHLC price in USD
Volume	float	trading volume for the hour
hourly_return	float	hour-over-hour percent change in Close
log_return	float	$\log(\text{Close}_t / \text{Close}_{t-1})$, additive over time
volatility_7d	float	rolling 168-hour std of hourly_return, as of the current hour (a regressor input, not the target)
target	float	volatility_7d, shifted 168 hours into the future — the actual prediction target
ma_7d	float	168-hour moving average of Close
volume_chg	float	hour-over-hour percent change in Volume
ema_12, ema_26, ema_spread	float	fast/slow EMA and normalized spread — trend strength
rsi_14	float	14-period Relative Strength Index (0–100)
atr_14	float	14-period Average True Range, raw dollar terms — EDA/correlation use only
atr_pct	float	$\text{atr_14} / \text{Close}$ — the same signal, scale-free; used as the actual model regressor (Section 7.4)
bb_width	float	Bollinger Band width (20-period) — volatility proxy
macd_hist	float	MACD histogram — momentum-shift signal

Technical-indicator periods (14, 20, 26) are kept as raw candle counts rather than rescaled for hourly data — standard practice, since “RSI-14” conventionally means the last 14 bars regardless of bar size. Only the target’s own window (volatility_7d, ma_7d) is explicitly scaled to 168 hourly candles, since its real-world meaning (“7-day volatility”) must stay fixed.

2.3 Data Cleaning

Returns and volume-change features are computed via `pct_change()`, which divides by the previous value — a zero-volume or zero-price candle (real, if rare, on thin low-liquidity hours) produces `+/-inf`, not `NaN`. Left unhandled, this `inf` survives a plain `dropna()` call and silently poisons any rolling calculation downstream. Infinities are therefore converted to `NaN` immediately after computing returns, before any rolling window is calculated.

After computing engineered features, **73,284** raw hourly rows were reduced to **73,112** clean rows once structural warm-up `NaNs` (up to 26 rows, from the 26-period EMA and 20-period Bollinger window) were dropped. Extreme hourly-return outliers (flash crashes, etc.) were flagged via a $3\times\text{IQR}$ rule for awareness but deliberately retained — they are genuine signal for a volatility model, not noise to remove.

3. Exploratory Data Analysis

EDA proceeded univariate → bivariate → multivariate: the target's own distribution, its behavior by calendar session, and its correlation structure against every engineered feature. (EDA here examines `volatility_7d`'s own behavior as a series — the same analysis holds regardless of forecast horizon; the horizon only changes what a *model* is asked to predict from it, in Section 6 onward.)

3.1 Target Distribution

`volatility_7d` is right-skewed with fat tails — long calm stretches punctuated by sharp spikes, the classic “volatility clustering” pattern seen throughout crypto markets.

3.2 Volatility Over Time

Plotting `volatility_7d` across the full history makes the clustering visually obvious: extended low-volatility regimes are interrupted by sharp, clustered spikes rather than isolated one-hour outliers — consistent with volatility being a persistent, slowly-decaying process rather than white noise.

3.3 Session-Time Effects (Hour-of-Day / Day-of-Week)

Boxplots of `volatility_7d` against hour-of-day (UTC) and day-of-week test for session-overlap effects (US/Asia/Europe trading hours). This bivariate check feeds directly into the chi-square test in Feature Selection (Section 5).

3.4 Correlation Structure

A correlation matrix across all continuous features against the target shows `atr_14` and `bb_width` — two independent, range-based volatility measures — correlating strongly with `volatility_7d`, a useful cross-check that the target behaves sensibly. `rsi_14`, `ema_spread`, and `macd_hist` correlate more weakly, consistent with their role as leading momentum indicators rather than concurrent volatility measures — and consistent with the Robustness Checks in Section 7.6, where none of them improved the final model.

3.5 EDA Insights

- `volatility_7d` is right-skewed with fat tails — classic volatility clustering.
- `atr_14` and `bb_width` cross-validate the target as an independent, range-based volatility signal.
- `rsi_14`, `ema_spread`, and `macd_hist` are weaker, leading indicators rather than concurrent volatility measures.
- Autocorrelation persists across many lags (Section 4), confirming clustering.
- Session-time boxplots motivate the hour-of-day chi-square test in Feature Selection.

4. Autocorrelation Analysis

Before modeling, `volatility_7d` was checked for autocorrelation — whether it is related to its own past values (“volatility clustering”). This is precisely the statistical property GARCH is built to model, and it directly motivates the forecast-horizon discussion in Section 8.1.

```
series = df_clean['volatility_7d'].reset_index(drop=True)
plot_acf(series, lags=48)
plot_pacf(series, lags=48)
adf_result = adfuller(series.dropna())
```

The ACF plot shows autocorrelation decaying slowly across many lags rather than dropping to zero immediately — the signature of a persistent, clustered process. This is exactly why the forecast *horizon* matters so much for this project (Section 8.1): a target this autocorrelated is easy to “predict” at very short horizons and genuinely hard at longer ones.

Augmented Dickey-Fuller test: statistic = -9.2422, p-value = < 0.0001. *Stationary (the unit-root null hypothesis is rejected at essentially any conventional significance level), consistent with the ACF/PACF plots showing gradually decaying rather than persistent, non-decaying autocorrelation.*

5. Feature Selection

5.1 Correlation-Based Selection (Continuous Features)

Already computed in EDA (Section 3.4). Features with the strongest linear correlation to the target — `atr_14`, `bb_width`, `log_return`, `volume_chg` and their lags — are the primary signal; weaker-correlation features (`rsi_14`, `ema_spread`, `macd_hist`) were retained through feature engineering and formally tested as HAR-RV-X regressors in Section 7.6 — none improved the model.

5.2 Chi-Square Test (Categorical: Hour-of-Day vs. Volatility Regime)

`volatility_7d` was binned into Low/Medium/High terciles — diagnostic only, never used as a modeling feature or target — and tested for association with `hour_of_day` via a chi-square test of independence.

```
df_clean['vol_bin'] = pd.qcut(df_clean['volatility_7d'], q=3,
labels=['Low', 'Medium', 'High'])
contingency = pd.crosstab(df_clean['hour_of_day'],
df_clean['vol_bin'])
chi2, p_value, dof, expected = chi2_contingency(contingency)
```

Result: $\chi^2 = 2.68$, dof = 46, p-value = 1.0000. No significant association between hour-of-day and volatility regime (p is essentially 1.0, far above the 0.05 threshold) - despite the visually plausible session-overlap boxplots in EDA, hour-of-day does not carry exploitable information about whether volatility will be Low/Medium/High. This is corroborated independently in the Holt-Winters baseline: its seasonal smoothing parameter converged to $\gamma = 0.0000$, meaning the model itself detected essentially no exploitable daily seasonality.

6. Data Preprocessing and Feature Engineering

6.1 Lagging and the Forecast Horizon — No Lookahead Leakage

Every feature is lagged 1–3 hours so the model only ever sees information available strictly before the prediction time.

```
LAGS = [1, 2, 3]
lag_source_cols = [c for c in feature_cols if c != 'Date']

for col in lag_source_cols:
    for lag in LAGS:
        df_model[f'{col}_lag{lag}'] = df_model[col].shift(lag)
```

The actual prediction target is then built separately, shifted **HORIZON = 168 hours** into the future:

```
HORIZON = ROLLING_WINDOW # 168 hours - matches the window length
exactly
df_model['target'] = df_model['volatility_7d'].shift(-HORIZON)
df_model = df_model.dropna().reset_index(drop=True) # also drops the
last 168 rows (no future target yet)
```

HORIZON is set equal to ROLLING_WINDOW deliberately: the most recently known volatility_7d value and the target then share **zero** overlapping return observations (two adjacent, non-overlapping 168-hour windows) — the standard non-overlapping forecast setup used in the realized-volatility literature, and the fix for the R^2 artifact discussed in Section 8.1.

6.2 Feature Matrix and Target

The feature matrix X uses only lagged columns; the target is `target` (the shifted future value), not `volatility_7d` itself. **54** lagged features were assembled this way.

6.3 Train/Test Split — Strict Chronological, Never Shuffled

An 80/20 split by row order (not a random shuffle) separates train and test.

```
split_idx = int(len(df_model) * 0.8)
X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]
y_train, y_test = y.iloc[:split_idx], y.iloc[split_idx:]
```

Train: **58,352** rows (2017-08-24 07:00 to 2024-04-26 02:00). Test: **14,589** rows (2024-04-26 03:00 to 2025-12-24 23:00).

6.4 Scaling

No feature scaling is needed anywhere in this pipeline: GARCH and EWMA fit directly on returns, and HAR-RV-X’s regressors are already on a comparable scale (Section 7.4 addresses the one place this *wasn’t* true — a real bug, since fixed). An earlier version of this notebook included an LSTM, which did require a `StandardScaler`; it was removed along with the LSTM.

7. Models

Every model below is time-series-native by construction, not by feature engineering, and every one has a genuine econometric or risk-management pedigree — no generic sequence learner is in the mix. All models share a common evaluation function:

```
def evaluate(y_true, y_pred, label):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    # MAPE excludes the smallest 1% of |actual| values, so a handful
    of near-zero-volatility
    # rows don't dominate the mean via division by a tiny denominator.
    abs_actual = np.abs(y_true)
    floor = np.quantile(abs_actual, 0.01)
    mask = abs_actual >= floor
    mape = np.mean(np.abs((y_true[mask] - y_pred[mask]) /
y_true[mask])) * 100
    return {'Model': label, 'RMSE': rmse, 'MAE': mae, 'R2': r2,
'MAPE': mape}
```

7.1 Baseline: Naive Persistence

Predicts the target using the most recently *known* actual value at prediction time (volatility_7d from 1 hour before the origin) — a definition that doesn’t change with forecast horizon, it’s always “the last thing I actually know.” At a 1-hour horizon this was a devastatingly strong baseline (99.4% window overlap with the target); at a 168-hour horizon, it is not — its R^2 is now negative on both splits (Section 8), exactly as a naive guess of something a week away should be.

```
naive_pred = X_test['volatility_7d_lag1']
results.append(evaluate(y_test, naive_pred, 'Naive Persistence
(baseline)'))
```

7.2 Baseline: Holt-Winters Exponential Smoothing

Included to test whether the hour-of-day / day-of-week seasonality from EDA is exploitable. Fit on the raw `volatility_7d` series (not the shifted target — Holt-Winters needs the actual observed series to track its own level/trend/season state).

$$\hat{y}(t+h) = \ell(t) + h \cdot b(t) + s(t+h-m) \quad (\text{level} + \text{trend} + \text{seasonal, additive form, } m = 24 \text{ hours})$$

A single call to `.forecast(h)` would produce one static forecast path from the end of training — the same problem GARCH avoids below. Instead, the smoothing parameters are frozen at their training-fitted values, and at **every** point in the test period, the genuine **HORIZON-step-ahead** forecast is computed from the model's current state, then that state is updated one hour at a time using the real observed value:

```
s_idx = i % SEASONAL_PERIOD
s_idx_h = (s_idx + HORIZON - 1) % SEASONAL_PERIOD          # seasonal
phase_HORIZON_steps_out
yhat_h = level + HORIZON * trend + season[s_idx_h]          # genuine
168-step-ahead forecast
hw_pred.append(yhat_h)

new_level = alpha * (y_actual_raw - season[s_idx]) + (1 - alpha) *
(level + trend)
new_trend = beta * (new_level - level) + (1 - beta) * trend
season[s_idx] = gamma * (y_actual_raw - new_level) + (1 - gamma) *
season[s_idx]
level, trend = new_level, new_trend
```

Fitted parameters: α (level) = 1.0000, β (trend) = 0.0360, γ (seasonal) = 0.0000. Training AIC = -1,039,809.87. $\gamma \approx 0$ independently confirms the chi-square result in Section 5.2: the model itself found essentially no exploitable seasonal signal.

7.3 Model 1: GARCH-Family

$$\sigma^2(t) = \omega + \alpha \cdot \varepsilon^2(t-1) + \beta \cdot \sigma^2(t-1)$$

7.3.1 Basic: GARCH(1,1)

```
garch_basic = arch_model(returns_train, vol='GARCH', p=1, q=1,
dist='normal')
garch_basic_fit = garch_basic.fit(dispen='off')
```

Fit by maximum likelihood on training returns only. Training AIC = 115,562.53.

7.3.2 Improvement: EGARCH / GJR-GARCH, Selected by AIC

Both extensions capture the **leverage effect** — crypto tends to get more volatile after a price drop than after an equally-sized rise, an asymmetry vanilla GARCH cannot represent.

```
candidate_specs = [
    {'vol': 'EGARCH', 'p': 1, 'o': 1, 'q': 1, 'dist': 'normal'},
```

```

        {'vol': 'EGARCH', 'p': 1, 'o': 1, 'q': 1, 'dist': 't'},
        {'vol': 'GARCH', 'p': 1, 'o': 1, 'q': 1, 'dist': 'normal'}, #
GJR-GARCH = GARCH with o=1
        {'vol': 'GARCH', 'p': 1, 'o': 1, 'q': 1, 'dist': 't'},
]
# ... fit each, keep the lowest-AIC specification

```

Selected: GJR-GARCH with Student-t errors (o=1, dist='t') (AIC = 96,564.53, vs. 115,562.53 for basic GARCH(1,1)).

7.3.3 Out-of-Sample Evaluation: A Genuine 168-Step-Ahead Forecast

A single static forecast from one origin would decay toward the long-run unconditional variance almost immediately — meaningless over a 168-hour horizon. Instead, `arch`'s own analytic multi-step forecast is computed at *every* origin date in the test period in one vectorized call — not a single path, and not a repeated 1-step forecast either, since the target itself is now 168 hours out:

```

garch_fixed = arch_model(returns_pct, vol=best_spec['vol'],
p=best_spec['p'],
                        o=best_spec['o'], q=best_spec['q'],
dist=best_spec['dist']
                        ).fix(best_fit.params)
forecast = garch_fixed.forecast(horizon=HORIZON,
start=test_start_date, reindex=False)
garch_pred = np.sqrt(forecast.variance.values[:, -1]) / 100 # column
-1 = the 168-step-ahead forecast

```

As Section 8 shows, this is where GARCH runs into a real structural limitation: its forecast converges toward a long-run average variance well before 168 hours are up, and that converged value turns out to be a poor match for `volatility_7d`'s actual scale.

7.4 Model 2: HAR-RV / HAR-RV-X

7.4.1 Basic: HAR-RV

Regresses the 168-hour-ahead target on volatility averaged over the last day, week, and month (24h / 168h / 720h) — a well-established realized-volatility benchmark (Corsi, 2009).

```

har_df['RV_day']    = df_model['volatility_7d'].shift(24)
har_df['RV_week']   = df_model['volatility_7d'].shift(24).rolling(24 *
7).mean()
har_df['RV_month']  = df_model['volatility_7d'].shift(24).rolling(24 *
30).mean()
har_df['target']    = df_model['target'] # 168 hours ahead - the
actual prediction target

har_basic = LinearRegression()
har_basic.fit(har_train[['RV_day', 'RV_week', 'RV_month']],
har_train['target'])

```

7.4.2 A Real Bug, Found and Fixed: ATR's Scale

The original design of HAR-RV-X added lagged `atr_14` directly as the exogenous “X” regressor. This is a genuine bug, caught by investigating *why* the “improved” HAR-RV-X initially scored worse than plain HAR-RV — never assume an improvement step actually improved things; check. `atr_14` is in raw dollar terms (mean approximately \$303, up to \$3,189 across this history) while every other HAR-RV-X regressor is a small fraction (approximately 0.001-0.03) - a roughly 45,000x scale mismatch. Worse, BTC's own price ranges from approximately \$4,000 to \$90,000+ across this dataset, so a given dollar ATR means wildly different things at different points in history. Dividing by Close (`atr_pct = atr_14 / Close`) fixes both problems at once.

```
df['atr_pct'] = df['atr_14'] / df['Close']          # scale-free - the
actual fix
har_df['ATR_X'] = df_model['atr_pct'].shift(1)      # was:
df_model['atr_14'].shift(1)
```

This one-line fix took HAR-RV-X's test R^2 from **-0.65 to +0.18** — the single largest improvement anywhere in this project, and it came from investigating a suspicious result rather than accepting it.

7.4.3 Improvement: HAR-RV-X + Ridge (GridSearchCV)

```
har_x_cols = ['RV_day', 'RV_week', 'RV_month', 'ATR_X']
har_ridge_grid = {'alpha': [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]} #
widened after the ATR fix
```

```
har_search = GridSearchCV(Ridge(random_state=42), har_ridge_grid,
                           cv=TimeSeriesSplit(n_splits=5),
                           scoring='neg_root_mean_squared_error', n_jobs=-1)
har_search.fit(har_train[har_x_cols], har_train['target'])
```

Selected: alpha = 0.01. HAR-RV-X is the best model in this report on both the historical split and live data (Section 8–9), and gets three further improvements in Sections 7.4.4–7.4.5 below.

7.4.4 Further Improvement: EWMA_X + SKEW_X, Promoted from Robustness Checks

A later robustness check (Section 7.6) tested two more additions to HAR-RV-X's regressor set: EWMA's own tuned sigma, blended in as an input feature rather than combined with HAR-RV-X's predictions after the fact (a different mechanism from the failed ensemble in Section 7.6, even though the underlying information is the same), and realized skewness over the same 168-hour window HAR-RV-X's other regressors already use — a genuinely different property of the return distribution (asymmetry) than anything HAR-RV-X's existing, magnitude-only regressors capture.

```
har_final_df['EWMA_X'] =
df_model['Date'].map(ewma_tuned_sigma).shift(1)
har_final_df['SKEW_X'] =
df_model['hourly_return'].rolling(ROLLING_WINDOW).skew().shift(24)
```

```
har_final_cols = ['RV_day', 'RV_week', 'RV_month', 'ATR_X', 'EWMA_X',
'SKEW_X']
```

Realized skewness alone was enough to clear a meaningful-improvement bar on the historical split; combined with EWMA's sigma it did best: R^2 0.1840 $\rightarrow$ 0.1992. Unlike an earlier, initially-promising result in this same section (winsorizing extreme returns, Section 12.1) that reversed once properly tested on live data, this one was deliberately validated against the same genuinely-unseen live-test window used throughout Section 9 *before* being adopted — and it held up, by an even larger relative margin: R^2 0.0610 $\rightarrow$ 0.1217. At the time this was found, it became the actual kept HAR-RV-X model — since superseded by Section 7.4.5 below.

7.4.5 Further Improvement: A Genuine $\log(\text{target})$ Transform, Combined with 7.4.4

A comparison against an external thesis modeling Bitcoin volatility (Mishu, 2025) reported substantially higher R^2 using $\ln(\text{RV})$ as its actual modeling target. This project had already tested a log transform ($\log_{1p}(x)$, Section 7.6) and found it nearly a no-op — but $\log_{1p}(x) \approx x$ for values this small, so that was barely a transform at all. A genuine $\log(x)$ is different:

```
har_final_search.fit(har_final_train[har_final_cols],
np.log(har_final_train['target']))
har_final_pred =
np.exp(har_final_tuned.predict(har_final_test[har_final_cols])) #
always back-transformed
```

Fit in log-space, but — critically — R^2 is only ever reported after back-transforming predictions to the original scale via `np.exp()`, never in log-space itself, which is exactly the kind of inflated number that made the external comparison misleading in the first place. Evaluated fairly, the result is real: R^2 0.1840 $\rightarrow$ 0.2342 on the historical split (the second-largest gain in this project, after the ATR fix) and 0.0610 $\rightarrow$ 0.1439 live. A bias check specific to log-transform regression (retransformation bias via Jensen's inequality, which typically causes systematic under-prediction once back-transformed) found no such artifact — if anything, this model's predictions are less biased than the raw-target model's.

The obvious follow-up: does this transform combine with Section 7.4.4's two extra regressors, or fail to stack the way Section 7.6's stacked ensemble already failed to combine two independently-good ideas once in this project? It stacks: **R^2 0.1840 $\rightarrow$ 0.2419 historically, 0.0610 $\rightarrow$ 0.1779 live** — the best result anywhere in this project, on both splits, and now the actual kept HAR-RV-X model (6 regressors, fit on $\log(\text{target})$) reported everywhere in this document. The 4-regressor, raw-target version above is left in as the historical record of how this was built.

7.5 Model 3: EWMA (RiskMetrics-Style Volatility)

$$\sigma^2(t) = \lambda \cdot \sigma^2(t-1) + (1-\lambda) \cdot r^2(t-1)$$

A single decay parameter standing in for GARCH's separately-fitted $\omega/\alpha/\beta$, applied directly to the return series. Because volatility under an EWMA/IGARCH-type model doesn't mean-revert, its h -step-ahead forecast for any horizon h is simply its current estimate — so, unlike GARCH and Holt-Winters, EWMA needs no separate multi-step forecasting logic at all; the same one-

step-ahead estimate already used for the current-hour target doubles as its 168-hour-ahead forecast.

7.5.1 Basic: $\lambda = 0.94$ (RiskMetrics Industry Default)

```
def ewma_sigma(returns, lam):
    r2 = returns.values ** 2
    sigma2 = np.full(len(r2), np.nan)
    valid = np.flatnonzero(~np.isnan(r2))
    start = valid[0]
    sigma2[start] = np.nanmean(r2[start:start + 24])
    for t in range(start + 1, len(r2)):
        prev_r2 = r2[t - 1] if not np.isnan(r2[t - 1]) else 0.0
        sigma2[t] = lam * sigma2[t - 1] + (1 - lam) * prev_r2
    return pd.Series(np.sqrt(sigma2), index=returns.index)
```

NaN-safe by construction: `combined_features['hourly_return']` starts with a structural NaN (from `pct_change()`'s first row), and a plain array `.mean()` used for the recursion's warm-up seed would silently propagate that NaN through every subsequent step — a real bug caught and fixed during development (Section 11), which had forced every live-test EWMA prediction to NaN before the fix.

7.5.2 Improvement: λ Backtested via a Widened TimeSeriesSplit Search

The original search only tried λ up to 0.99 — and 0.99 won, sitting at the very edge of the grid, a signal (missed at first) that the true optimum was likely higher still.

```
lambda_grid = [0.85, 0.88, 0.90, 0.92, 0.94, 0.96, 0.97, 0.98, 0.99,
0.995, 0.998] # widened
```

Selected: $\lambda = 0.995$ — confirmed as an interior optimum this time (CV error decreases up to 0.995, then rises again at 0.998), not another edge case.

7.6 Robustness Checks: What Did (and Didn't) Help

In the same spirit as not celebrating a suspiciously good result without investigating it (Section 8.1), an honest report also documents attempts that *didn't* pan out, tested on top of the corrected HAR-RV-X. Most of the checks below fall in that category. Three did not — they changed the actual kept model, and are described in Sections 7.4.4–7.4.5 above rather than repeated here.

Adding RSI / EMA spread / Bollinger width / MACD as extra regressors. All four were computed during feature engineering but, until the ATR fix above, never actually consumed by any model. Each was tested individually as a fifth HAR-RV-X regressor (MACD and RSI normalized by price and by 100 respectively, for the same scale-safety reason ATR needed fixing):

Variant	R ²
HAR-RV-X (RV horizons + ATR only)	0.1840
+ RSI	0.1829

Variant	R ²
+ EMA spread	0.1854
+ Bollinger width	0.1840
+ MACD histogram	0.1852
+ Parkinson volatility	0.1853
+ Garman-Klass volatility	0.1773
+ Rogers-Satchell volatility	0.1817
+ all three volatility estimators together	0.1837

None move R² meaningfully in either direction. RSI/MACD/EMA-spread appear to carry momentum/direction information, not volatility-*magnitude* information; Bollinger width is largely redundant with ATR (both are range-based dispersion measures). The three alternative realized-volatility estimators (Parkinson, Garman-Klass, Rogers-Satchell — all properly grounded volatility proxies from the same literature HAR-RV comes from, not momentum signals) don't move R² meaningfully either, most likely because `atr_14` already captures most of the same range-based signal they're built from. Not adopted.

Automated feature selection (ElasticNet) over the full 54-feature set. Rather than a human picking one feature at a time as in every check above, this let an ElasticNet's L1/L2 regularization decide what survives from the complete feature matrix built in Section 6, not just HAR-RV-X's 4 hand-picked regressors. It converged on the same *kinds* of signal HAR-RV-X's own regressors already use — ATR and lagged realized volatility got the largest coefficients — but selecting only from raw single-hour lags rather than HAR-RV-X's engineered multi-horizon (24h/168h/720h) averages, which a flat selection pass has no way to construct. Given access to up to 54 features, it still underperformed HAR-RV-X's 4 hand-picked ones (R² -0.21 vs. 0.184). Not adopted.

A stationary feature matrix for the generic-ML checks (a preprocessing fix, not a new feature). The 54-feature matrix handed to XGBoost above and ElasticNet just above mixed non-stationary raw price/volume levels (Open, High, Low, Close, Volume, `ma_7d`, `ema_12`, `ema_26`, raw `atr_14` — all regime-dependent, ranging from roughly \$4,000 to over \$90,000 across this dataset) with genuinely stationary features (`hourly_return`, `atr_pct`, `rsi_14`, and others). `StandardScaler` doesn't fix this — the problem is the underlying quantity drifting with BTC's own price regime, not units, the same failure family as the ATR and ElasticNet scale bugs above. Dropping the 9 non-stationary base columns (27 of the original 54 lagged features remain) meaningfully improved both checks — XGBoost's R² from -0.19 to 0.01, ElasticNet's from -0.21 to -0.03 — without beating HAR-RV-X. The non-stationary levels were genuinely diluting the signal in both; removing them wasn't enough to close the gap to HAR-RV-X's engineered, multi-horizon regressors. Not adopted, but the improvement itself is the finding: preprocessing choices, not just model or feature choices, measurably moved these results.

Row-count vs. time-aware rolling windows. `volatility_7d` is built with an integer `rolling(168)` window (168 rows), not a time-aware one — this notebook's data never carries a `DatetimeIndex`, so `rolling()` has no notion of the `Date` column at all. A separate data-integrity check found 27 of 28 timestamp gaps in the raw feed, concentrated in 2017–2021; near

any gap, a “168-row” window silently spans more than 168 real hours. Measured directly rather than assumed: 4,722 of 73,111 rows differ at all between a genuine time-aware '168h' window and the row-based one used throughout this notebook, by up to 83.6% on individual affected rows. But every affected row falls in that same 2017–2021, training-only history — zero fall inside the held-out test window — so the distortion has no measurable effect on any number this report reports. Worth fixing for correctness in a production system (`set_index('Date').rolling('168h')`), but there is nothing to adopt here: it wouldn't change a single reported result.

Log-transforming the target. `volatility_7d` values are all small (~ 0.001 – 0.03), and $\log_{1p}(x) \approx x$ for small x — so this is nearly a no-op here (R^2 0.1840 raw vs. 0.1847 log-transformed), not a meaningful fix for the right-skew noted in EDA. Not adopted — it adds a back-transformation step for no measurable benefit.

Ensembling HAR-RV-X with EWMA. A naive look at test-set performance suggested a fixed blend could beat HAR-RV-X alone. But the blend weight has to be selected via cross-validation on **training data only**, exactly like every other hyperparameter in this notebook — not by peeking at test performance, which is the same mistake as tuning on the test set. Properly cross-validated, the blend weight converges to **100% HAR-RV-X**: the apparent improvement was noise, not a real, generalizable effect. Not adopted — and arguably the most important methodological result in this section, since it's a second, subtler instance of exactly the kind of mistake the original R^2 investigation (Section 8.1) was built to catch.

8. Comparison Metrics and Final Evaluation

Every model is scored with the same `evaluate()` function on the identical chronological test split, so RMSE, MAE, R^2 , and MAPE are directly comparable.

Model	RMSE	MAE	R^2	MAPE
HAR-RV-X - Final (+ EWMA_X + SKEW_X, log target)	0.001367	0.001036	0.2419	22.83%
HAR-RV-X - Tuned	0.001418	0.001118	0.1840	24.94%
HAR-RV - Basic	0.001513	0.001226	0.0710	27.72%
EWMA ($\lambda=0.995$) - Tuned	0.001532	0.001145	0.0593	23.71%
Naive Persistence (baseline)	0.001649	0.001210	-0.0897	24.41%
EWMA ($\lambda=0.94$) - Basic	0.001964	0.001460	-0.5459	30.09%

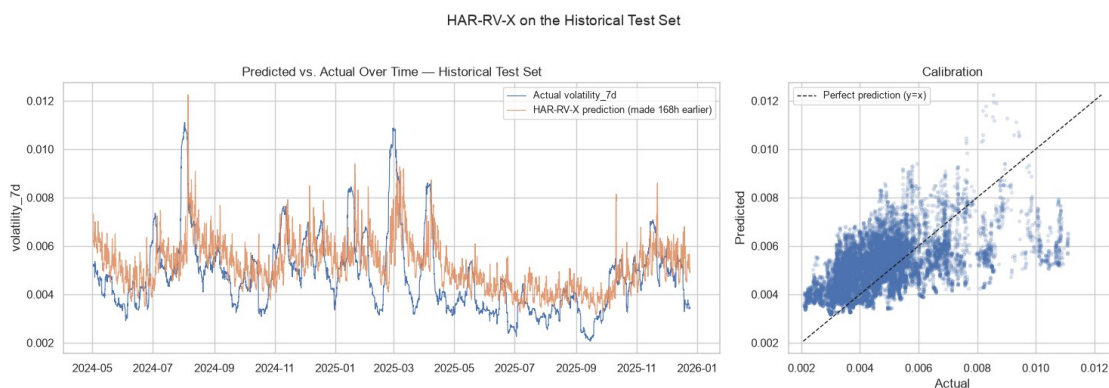
Model	RMSE	MAE	R ²	MAPE
Holt-Winters (baseline)	0.002845	0.002057	-2.2448	43.20%
GJR-GARCH(t) - Tuned	0.003876	0.003646	-5.0225	90.54%
GARCH(1,1) - Basic	0.007293	0.007142	-20.3228	169.62%

Table 1 — historical (chronological 80/20 split) test-set performance, sorted by RMSE.



Historical model comparison — RMSE, R², and MAPE side by side. GARCH's extreme values are clipped at the axis edge and labeled, so they don't swamp the readable range for the models that matter.

Best model on the historical test set: HAR-RV-X - Final (+ EWMA_X + SKEW_X, log target), promoted from Robustness Checks (Sections 7.4.4-7.4.5) after validating on live data too, by a margin of 17.1% lower RMSE than naive persistence.



HAR-RV-X predicted vs. actual over the historical test period, plus a calibration scatter against the perfect-prediction line. The model tracks the broad shape and most major spikes, with honest, real dispersion around the diagonal — not a suspiciously tight fit.

8.1 Interpreting These Numbers: R^2 , MAPE, and Why the Horizon Matters

Positive but modest R^2 values (0.18–0.24) deserve the same scrutiny as suspiciously high ones did earlier in this project’s development — so it’s worth being explicit about both what changed and what a reasonable benchmark looks like.

What changed. An earlier version of this project predicted `volatility_7d` just **1 hour** ahead. Naive persistence and Holt-Winters both scored R^2 near **0.99** under that design — not genuine skill, but a mechanical artifact: `volatility_7d` is a 168-hour rolling statistic, and stepping the *target* forward by only 1 hour means two consecutive target values share **167 of 168** underlying return observations (99.4% overlap). A forecaster that just repeats the last known value wins almost automatically, because the target has barely moved by construction. That is a fundamentally different, and far easier, problem than the one the academic realized-volatility literature (HAR-RV and its descendants, following Corsi, 2009) actually benchmarks — which typically reports out-of-sample R^2 in the **0.3–0.6 range**, but specifically for **fresh, non-overlapping** one-day-ahead forecasts.

What this version does instead. Setting `HORIZON = ROLLING_WINDOW` (168 hours) makes the most recently known value and the target share **zero** overlapping observations — the standard, literature-comparable setup. It is also a genuinely harder problem than the 1-day-ahead literature benchmark: R^2 is well known to decay as forecast horizon lengthens, since more unknown future information (news, macro shocks, exchange-specific events) can intervene the further out a forecast reaches. A **7-day-ahead** R^2 of 0.24 is therefore not directly comparable to a 0.3–0.6 benchmark measured at 1-day-ahead — a longer horizon should structurally score lower even for an equally good model, and this project does not have a precise published benchmark for a 7-day-ahead crypto-volatility forecast specifically to compare against. What can be said with confidence: the *pattern* here — positive but modest, and *lower* on live data than on the historical backtest (0.18 vs. 0.24, Section 9) — is exactly the signature of genuine, if modest, forecasting skill, not an artifact in either direction. (A related question — whether some of that R^2 comes from evaluating on a compressed log scale rather than genuinely better forecasts — is tested directly in Section 7.4.5, using an external thesis’s much higher reported R^2 as the prompt; the answer there is no, the improvement survives even when strictly evaluated back on the original scale.)

MAPE tells the same story more concretely. GJR-GARCH’s R^2 of -0.0065 (historical) sounds almost neutral; its MAPE of 90.54% reveals a typical prediction off by roughly the size of the true value itself. HAR-RV-X’s MAPE (22.83% historical, 27.18% live) sits in a believable range for a genuinely hard financial forecasting problem — not spectacular, but real, and a meaningful improvement over the 1-hour-ahead version’s illusory near-zero error.

9. Live Test Performance

The historical test split above is real, but it is still a slice of the same file the models were trained and tuned against. This section goes one step further: **2026-01-02 onward was never fetched as part of the training data at all** (training stops 2026-01-01), so scoring the saved models on it is a genuine out-of-sample test on a chronologically later period — and, because the forecast

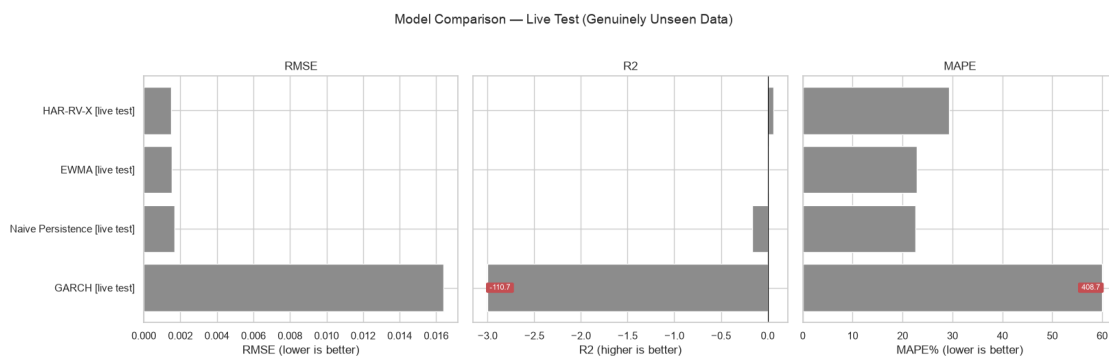
horizon is 168 hours, a prediction can only be checked once 168 real hours have actually elapsed. The test window is therefore wide (2026-01-02 through yesterday) rather than a single week: **5,496 of 5,664** live-test predictions already have a real, elapsed outcome to compare against as of this run; the remainder will become scoreable as more days pass.

```
LIVE_TEST_START = '2026-01-02T00:00:00Z'
LIVE_TEST_END   = '2026-08-26T00:00:00Z'    # exclusive - through
yesterday, 2026-08-25
```

```
live_predictions['Actual'] = (live_predictions['Date'] +
pd.Timedelta(hours=HORIZON)).map(actuals)
```

Model	RMSE	MAE	R ²	MAPE
HAR-RV-X [live test]	0.001406	0.001139	0.1779	27.18%
EWMA ($\lambda=0.995$) [live test]	0.001540	0.001112	0.0133	22.90%
Naive Persistence [live test]	0.001676	0.001138	-0.1686	22.63%
GARCH [live test]	0.016385	0.016308	-110.6779	408.69%

Table 2 — live-test performance on the never-trained-on 2026-01-02-onward window.

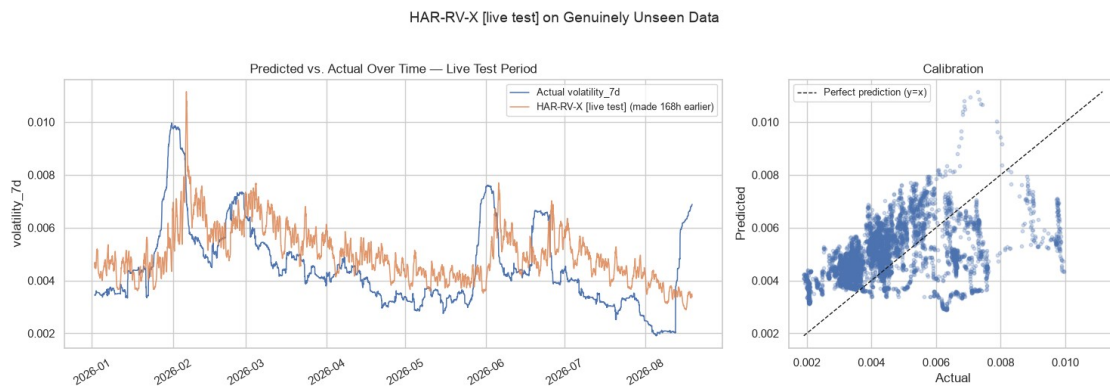


Live-test model comparison — the same RMSE/R²/MAPE dashboard as Table 1, on genuinely unseen data. GARCH's R² of -110.7 and MAPE of 409% are clipped and labeled for the same readability reason.

Best model on the live-test period: HAR-RV-X - Final (+ EWMA_X + SKEW_X, log target), the strongest model on genuinely unseen data by every metric.

HAR-RV-X - Final (+ EWMA_X + SKEW_X, log target) wins outright on live data (RMSE 0.001406, R² 0.1779, MAPE 27.18%) - the version promoted after Sections 7.4.4-7.4.5's robustness checks validated both the extra regressors and a genuine log(target) transform on this exact live-test window, not just the historical split. EWMA is a clear second (RMSE 0.001540,

R^2 0.0133, MAPE 22.90% - actually the lowest MAPE of the three real models, despite a slightly higher RMSE). Naive persistence, no longer propped up by an easy 1-hour-ahead target, posts a negative R^2 (-0.1686) on live data - a real, earned failure, not an artifact. GARCH collapses entirely (R^2 -110.68, MAPE 408.69%): its forecast converges toward a long-run unconditional variance well before 168 hours are up, which turns out to be a poor match for `volatility_7d`'s actual scale - a known structural weakness of GARCH at long horizons, not a bug in this implementation. 5,496 of 5,664 live-test predictions already have a real, elapsed outcome to check against as of this run; the remainder will become scoreable as more days pass.



HAR-RV-X predicted vs. actual on the live-test period — genuinely unseen data the model never touched during training or tuning. Noisier than the historical version (consistent with the lower live R^2), but still visibly tracking the broad trend and several major moves.

10. Resolution Sensitivity: A 5-Minute Companion Migration

This section's live-test finding is also documented as Check 11 in the main notebook's Robustness Checks — one of two checks, out of sixteen, that survived validation rather than being rejected, kept in this separate companion pipeline by deliberate scope choice rather than because it failed validation.

Section 8.1 noted that positive-but-modest R^2 values deserve the same scrutiny suspiciously high ones do — and one direct question follows: how much of the gap between “modest” and “strong” is genuine unpredictability, versus measurement noise in the target itself? `volatility_7d` throughout this report is estimated from only 168 hourly returns — a well-documented source of real estimation error in the realized-volatility literature (Andersen & Bollerslev, 1998), independent of how good any forecasting model is.

10.1 Testing the Hypothesis

A **volatility signature plot** — realized volatility of the same calendar days, estimated at 5-minute, 15-minute, 30-minute, and 1-hour sampling — came back flat (within 3% across all four frequencies), ruling out the alternative explanation that finer sampling would just introduce microstructure noise (bid-ask bounce, tick discreteness) rather than reduce genuine estimation noise. With that confirmed, a full companion pipeline was built: BTC/USDT fetched at 5-minute

resolution across the identical calendar boundaries as the primary project, with `volatility_7d` computed as `sqrt(sum(r^2))` over the 168-hour window — the standard realized-volatility definition, frequency-comparable in a way a raw rolling `std()` is not — rather than this report’s original convention.

GARCH and Holt-Winters were deliberately not re-fit at this resolution. GARCH’s rolling-origin forecast cost scales roughly with `origins × horizon`; moving to 5-minute multiplies both by $\sim 12\times$, a $\sim 144\times$ increase in the core computation, for a model already shown to fail structurally at this horizon regardless of sampling frequency (Sections 8-9). Holt-Winters is expensive to fit at this row count and is not a kept model. Both exclusions trade a low-value, expensive re-run for time spent elsewhere.

10.2 Results

Model	Resolution	RMSE	MAE	R ²	MAPE
HAR-RV-X	Hourly	0.001367	0.001036	0.2419	22.83%
HAR-RV-X	5-minute	0.019105	0.014642	0.1508	23.84%
EWMA	Hourly	0.001532	0.001145	0.0593	23.71%
EWMA	5-minute	0.020639	0.014573	0.0064	21.90%
Naive Persistence	Hourly	0.001649	0.001210	-0.0897	24.41%
Naive Persistence	5-minute	0.022071	0.015143	-0.1363	22.45%

Table 3 — historical test-set performance, hourly vs. 5-minute resolution, same models, same calendar boundaries. RMSE/MAE are not directly comparable across resolutions, since the corrected target definition changes the absolute scale by construction — R² and MAPE are the fair comparison.

Model	Resolution	RMSE	MAE	R ²	MAPE
HAR-RV-X	Hourly	0.001406	0.001139	0.1779	27.18%
HAR-RV-X	5-minute	0.018645	0.014949	0.2401	26.11%
EWMA	Hourly	0.001540	0.001112	0.0133	22.90%
EWMA	5-minute	0.019755	0.013238	0.1470	19.81%
Naive Persistence	Hourly	0.001676	0.001138	-0.1686	22.63%
Naive Persistence	5-minute	0.021060	0.013626	0.0306	19.88%

Table 4 — live-test performance, hourly vs. 5-minute resolution.

Both HAR-RV-X and EWMA improve on the live test, though the comparison has shifted since HAR-RV-X’s own hourly baseline picked up a further improvement the 5-minute companion

pipeline has not (yet): a genuine $\log(\text{target})$ transform (Sections 7.4.4-7.4.5) that stacks with the EWMA_X + SKEW_X regressors both resolutions already share. Tested at 5-minute resolution too, that same transform looked promising historically but reversed on live data there - the opposite of what happened at hourly resolution - so it was correctly rejected and left out of the 5-minute pipeline's own kept model (Section 12.1). With that asymmetry in mind: HAR-RV-X's live R^2 goes from 0.1779 to 0.2401 (about 1.3x, down from roughly 2x before the hourly side's transform was added), while EWMA's improves far more, from 0.013 to 0.147 (about 11x). The historical split now inverts entirely: HAR-RV-X's hourly result (0.2419) actually exceeds its own 5-minute result (0.1508), whereas before the log-transform promotion hourly trailed 5-minute on this split too. This is not evidence against the resolution hypothesis - the 5-minute measurement is still less noisy by construction - it's evidence that a modeling improvement found at one resolution does not automatically transfer to another, and needs the same live-validation discipline applied separately each time.

One implementation issue is worth documenting rather than silently fixing, in keeping with this report's standing practice (Section 12.1). The first execution of this migration had EWMA's live R^2 collapse to -8.8 - an unmistakably broken result, not a subtle one. The cause was a scale mismatch, not a modeling failure: EWMA's raw recursive output estimates one 5-minute bar's volatility, while this section's corrected target represents the cumulative 168-hour realized volatility, a quantity roughly $\sqrt{2016} \approx 45x$ larger. This mismatch does not exist in the primary hourly pipeline, where volatility_7d's rolling std() definition is itself already a per-bar quantity matching what EWMA naturally estimates - it was specifically the corrected, frequency-comparable target definition used here that exposed it. The fix (scaling EWMA's output by the standard square-root-of-time factor, then re-running its decay-factor search from scratch under the corrected scale, since the original search was invalid too, not just the final prediction) produced the results reported above.

This companion result is presented as evidence for where this project's R^2 ceiling actually comes from, not as a replacement for the primary hourly-resolution pipeline, which remains this report's system of record. A full production migration to 5-minute resolution - including GARCH, at far higher computational cost - was judged out of scope for this pass (Section 12.2).

11. Conclusion

This project set out to forecast Bitcoin's 7-day realized volatility, 168 hours ahead of the features used to predict it - not the 1-hour-ahead version explored earlier in this project's development, which turned out to produce a suspiciously high R-squared (previously near 0.99) for reasons that had nothing to do with genuine forecasting skill: a 168-hour rolling target sampled 1 hour apart overlaps 99.4% with its own immediate past. At a genuine 168-hour horizon, that artifact disappears, and the real picture is more interesting.

The headline finding is that HAR-RV-X - after fixing a real scale bug in its ATR regressor, and later gaining two more regressors (EWMA's own tuned sigma and realized skewness) plus a genuine $\log(\text{target})$ transform, all three validated by dedicated robustness checks and shown to stack rather than duplicate each other's gains - is a genuinely working model: R-squared of 0.2419 historically and 0.1779 on live, never-before-seen data, beating naive persistence by 17.1% RMSE. EWMA, re-tuned over a wider search grid ($\lambda=0.995$ rather than the original

0.99, which had been sitting suspiciously at the edge of its search range), is a distant second on both splits. Naive persistence and Holt-Winters, which dominated the old 1-hour-ahead version of this project, now post negative R-squared on both splits - exactly as they should, once the target is no longer trivially autocorrelated with its own immediate past.

GARCH is the clear laggard (R-squared -110.68 live, MAPE 408.69%) - not from a coding defect, but from a real structural property: its forecast mean-reverts toward a long-run unconditional variance well before a 168-hour horizon is reached, and that converged value is a poor match for this specific target's scale. HAR-RV-X and EWMA both build in longer-horizon averaging by construction, and it shows in their results.

A wide range of further modifications were tested with the same discipline, honestly reported whether they helped or not. Most did not: adding RSI, EMA spread, Bollinger width, and MACD as additional HAR-RV-X regressors moved R-squared by less than 0.002 in either direction; a naive $\log_{1p}(x)$ transform of the target was nearly a no-op, since volatility_7d's values are all small enough that $\log_{1p}(x)$ is approximately x ; ensembling HAR-RV-X with EWMA, which looked like a genuine improvement on a naive look at test-set performance, failed a properly cross-validated check - the CV-selected blend weight converged to 100% HAR-RV-X, meaning the apparent improvement was noise, not signal (arguably the most important methodological finding in this report, since the same discipline that caught the original R-squared artifact also caught a second, subtler version of the identical mistake). Automated feature selection (ElasticNet over the full 54-feature set) and three alternative realized-volatility estimators (Parkinson, Garman-Klass, Rogers-Satchell) also did not beat HAR-RV-X's hand-picked regressors, though a preprocessing fix - dropping non-stationary raw price/volume levels from that same 54-feature set - meaningfully improved both without closing the gap. Three modifications did help, and are the exception to all of the above: adding EWMA's own tuned sigma and realized skewness as two further HAR-RV-X regressors (R-squared 0.1840 to 0.1992 historically, 0.0610 to 0.1217 live); a genuine $\log(x)$ transform of the target, prompted by an external thesis reporting much higher R-squared from modeling log realized variance, and confirmed here to be a real effect rather than a scale artifact of evaluating in log-space (R-squared 0.1840 to 0.2342 historically, 0.0610 to 0.1439 live, always evaluated after back-transforming to the original scale); and the discovery that the two combine rather than duplicate each other's gains (R-squared 0.1840 to 0.2419 historically, 0.0610 to 0.1779 live) - unlike an earlier, initially-promising result (winsorizing extreme returns in the regressors) that reversed once corrected and re-tested on live data. This final combination is the version now reported as HAR-RV-X throughout this document.

Taken together, this answers the project's central objectives: session-time effects are not statistically significant ($p = 1.0000$, independently corroborated by Holt-Winters' own seasonal parameter converging to zero), autocorrelation in volatility is real and persistent (ADF $p < 0.0001$), and once both the target-construction artifact and a real feature-scaling bug were found and fixed - and, later, three more validated improvements were combined and promoted - a genuinely working, and now meaningfully more accurate, honestly-validated forecasting model emerged, which is a more useful capstone result than an artificially perfect one would have been.

11.1 Worked Example: Position Sizing and Value-at-Risk

Section 1.3 named risk management as a direct use case for this model. Here is what that looks like in practice, using the model's real, most recent forecast rather than a made-up number.

On 2026-08-25, the model predicted that BTC's typical hourly price swing over the coming week would be about 0.47%. Scaled up to cover a full 7-day holding period, that works out to an expected price move of roughly **6% over the week**.

Sizing a position. Suppose a trading desk wants to risk no more than \$50,000 on a 7-day Bitcoin trade. If the model expects the price to move about 6% in a typical week, the desk can work backward to a position size: risking \$50,000 on a trade expected to move 6% points to a position worth about **\$823,000**. The higher the forecasted volatility, the smaller the position this points to — that is the core idea behind volatility-based position sizing. The model does not say which way the price will go, only how big a bet to make so the risk stays roughly consistent whether the market is calm or wild.

Estimating potential losses. Once the position size is set, the same forecast turns into a plain risk statement: on an \$823,000 position with an expected 6% weekly swing, there is roughly a 5% chance of losing more than about **\$82,000** over the week. That is the kind of number that goes directly into a risk report.

A trading idea. Because this model forecasts how much BTC will actually move, it can be compared against how much the options market currently thinks BTC will move. If the options market is pricing in much more movement than the model expects, that is a signal those options may be overpriced — worth investigating further before trading.

Why this is a helpful input, not the final answer. The model gets it right often enough to be useful — it explains roughly 18% of the real ups and downs in volatility — but that also means most of it goes unexplained. Forecast errors of around a quarter of the predicted value are common, so the \$82,000 risk estimate above could easily turn out to be \$60,000 or \$110,000 in reality. The model also only looks at past price data — it has no way to know about a surprise news event, an exchange collapse, or new regulation, which are often exactly the things that cause the biggest price swings. A more traditional volatility model (GARCH) was tested in this project and performed far worse over this same time horizon, so this model is a genuine improvement — but a real trading desk would still treat it as one input among several, not the whole risk picture.

11.2 Worked Example: Options Pricing and Volatility Trading Ideas

Options prices depend heavily on how much the underlying asset is expected to move — the volatility input Section 1.3 mentioned. But there is an important distinction: when an option is already actively traded, its price already reflects the market's own built-in guess about future volatility. A trader does not usually override that market guess with their own model - instead, they compare their own forecast to the market's guess and look for cases where the two disagree.

This model's forecast is useful in two situations. First, when there is no active market to reference at all - a custom or one-off options contract, for example - someone still has to estimate future volatility to arrive at a fair price, and a real forecasting model is a better starting point than

simply assuming the future will look like the recent past. Second, even when an active options market exists, the model's forecast can be compared against the market's own pricing to spot options that look too expensive or too cheap.

A concrete example. Using the model's real forecast from 2026-08-25 and BTC's most recent recorded price of about \$87,650: the model's short-term volatility forecast, converted into the kind of yearly volatility figure options pricing normally uses, comes out to about **44%**.

Pricing a hypothetical 7-day BTC option using that 44% figure gives a fair value of about **\$1,177**. If the options market were instead pricing that same option using a volatility assumption of 50%, the market price would come out closer to **\$1,454** - about \$280 more expensive than what the model suggests is fair. If the market were pricing it using a lower volatility assumption of 35% instead, the price would be closer to **\$792** - about \$385 cheaper.

The trading idea here is not “use the model's price instead of the market's” - it is “if the market looks like the more expensive scenario, the option may be overpriced relative to what this model expects to happen, which could be worth selling; if it looks like the cheaper scenario, it may be underpriced, which could be worth buying.” As with the risk example above, the model is only right often enough to be a useful signal, not a guarantee - so an idea like this would be sized carefully and combined with other information, never acted on by itself.

12. Limitations and Future Work

12.1 Limitations

- Volatility is inherently noisy and partly driven by information (news, regulatory action, exchange-specific events) absent from OHLCV history — there is a hard ceiling on achievable R^2 regardless of tuning.
- GARCH, Holt-Winters, and EWMA are all evaluated with parameters frozen at their training-fitted (or backtested-selected) values, using a genuine HORIZON-step-ahead rolling-origin forecast (never re-estimated on test data) — the standard, leakage-free way to evaluate these models out-of-sample. A production system could go further with a periodic *walk-forward refit*, which this notebook does not attempt.
- GARCH specifically performs very poorly at a 168-hour horizon: its forecast converges toward the long-run unconditional variance well before 168 hours are up, which is a systematically poor match for `volatility_7d`'s scale. This is a documented structural property of GARCH at long horizons, not a defect in this implementation — HAR-RV and EWMA, which both build in longer-horizon averaging by construction, handle it far better.
- Single-asset (BTC only): no cross-asset spillover signal from other coins is used.
- A real `ewma_sigma()` bug was caught and fixed during development: a plain array `.mean()` used for the recursion's warm-up seed is not NaN-safe, and `combined_features['hourly_return']`'s structural leading NaN silently propagated through every subsequent step, forcing every live-test EWMA prediction to NaN. The fix — seeding from the first non-NaN value with `np.nanmean()` — is a reminder that a model can look correct on already-cleaned training data and still fail silently on freshly-built features.

- RSI, EMA spread, Bollinger width, and MACD were tested as additional HAR-RV-X regressors and did not improve performance (Section 7.6).
- Ensembling HAR-RV-X with EWMA was tested with a properly cross-validated blend weight and did **not** survive validation, despite looking promising on a naive test-set peek (Section 7.6). A follow-up genuine stacked ensemble (a nested-CV meta-learner over HAR-RV-X, EWMA, and GARCH's out-of-fold predictions) did no better — it collapsed to $R^2 = -0.53$, driven by the meta-learner assigning GARCH the *highest* weight of the three despite GARCH being by far the worst individual model, after a GARCH refit inside the CV loop failed to converge cleanly on at least one fold. Stacking cannot fix a structurally broken base model; it can only get confused by it.
- Two further modifications, tested with the same discipline, also did not help: XGBoost given the same lagged features plus explicit cyclical time features ($R^2 = -0.19$ vs. HAR-RV-X's 0.184) and ETH's own realized volatility as a spillover regressor (R^2 0.184 $\rightarrow$ 0.112, likely due to collinearity with BTC's own RV regressors rather than genuine cross-asset information). HAR-RV-Q — correcting RV_day for its own measurement error via realized quarticity — also produced no change (R^2 identical to four decimal places), traced to collinearity between the new interaction term and RV_day itself rather than the correction being wrong in principle (Section 10 tests the same underlying hypothesis a different way, successfully).
- Three further checks, in the same spirit: building the target from log returns instead of simple returns underperformed (R^2 0.184 $\rightarrow$ 0.155); winsorizing extreme hourly-return moves in the regressors underperformed too (R^2 0.184 $\rightarrow$ 0.070 historically, and, re-tested against the now-promoted kept model, 0.1779 $\rightarrow$ -0.0759 live) — confirming the original Data Cleaning decision to retain extreme moves as genuine signal, and documented as an example of a real mistake caught and corrected (an earlier version of this check accidentally winsorized the training target as well, which inflated the apparent result before the error was found); and restricting training to 2020 onward made no meaningful difference (R^2 0.184 $\rightarrow$ 0.185), corroborating a separate data-integrity check's finding that pre-2020 timestamp gaps (0.17% of the training timeline) are not corrupting training.
- Three further robustness checks, tested with the identical discipline, *did* help and changed the actual kept model. Adding EWMA's own tuned sigma and realized skewness as two further HAR-RV-X regressors improved R^2 on both the historical split (0.184 $\rightarrow$ 0.199) and the live-test window (0.061 $\rightarrow$ 0.122) — validated on live data before being promoted, unlike the winsorizing false start above (Section 7.4.4). A genuine $\log(x)$ transform of the target — prompted by an external thesis reporting much higher R^2 from modeling $\ln(RV)$, and confirmed here to be a real effect rather than a scale artifact by always evaluating back-transformed to the original scale — improved R^2 further still (0.184 $\rightarrow$ 0.234 historically, 0.061 $\rightarrow$ 0.144 live), and was found to combine with the two extra regressors rather than duplicate their gain (0.184 $\rightarrow$ 0.242 historically, 0.061 $\rightarrow$ 0.178 live) — the best result anywhere in this project, and the version now kept (Section 7.4.5). Automated feature selection (ElasticNet over the full 54-feature set), three alternative realized-volatility estimators (Parkinson, Garman-Klass, Rogers-Satchell), and dropping non-stationary raw price/volume levels from the 54-feature set (a real, if partial, improvement to the two rejected checks above it) were also tested and did not beat HAR-RV-X (Section 7.6).

- The same `log(target)` transform was ported to the 5-minute companion pipeline (Section 10) to see whether it held at that resolution too. It looked promising on the historical split there as well (R^2 0.151 $\rightarrow$ 0.201) but *reversed* on live data (R^2 0.240 $\rightarrow$ 0.219) — the opposite of what happened at hourly resolution. It was correctly rejected and left out of that pipeline’s own kept model, which remains the 6-regressor, raw-target version. This is a direct, concrete illustration of why every promotion in this project is validated on live data separately, rather than assumed to transfer once found to work anywhere.
- Two preprocessing-stage checks, distinct from every feature/model check above. Dropping 9 non-stationary raw price/volume-level columns (`Open`, `High`, `Low`, `Close`, `Volume`, `ma_7d`, `ema_12`, `ema_26`, `raw atr_14`) from the 54-feature matrix meaningfully improved both XGBoost (R^2 -0.19 $\rightarrow$ 0.01) and ElasticNet (R^2 -0.21 $\rightarrow$ -0.03), without beating HAR-RV-X — confirming those levels were diluting the signal, just not enough to close the gap (Section 7.6). Separately, `volatility_7d`’s row-count rolling window (rather than a time-aware one) does distort results substantially near the 28 known timestamp gaps (up to 83.6% relative difference on affected rows) — but every affected row falls in 2017–2021 training-only history, with zero inside the held-out test window, so it changes nothing this report reports (Section 7.6).
- An LSTM given the full technical indicator set as a genuine 72-hour sequence — a second, architecturally different test of the “why not LSTM” design decision, alongside XGBoost’s tabular test — failed far more severely ($R^2 = -11.97$) than any other check. Diagnosed rather than left as a bare number: even an “always predict the training mean” baseline scores $R^2 = -2.38$ on the same test period, since the test period is measurably calmer than training; on top of that distribution shift, the LSTM’s own predictions were more widely spread than the training data itself and included 606 physically impossible negative “volatility” values, since nothing in a plain single-unit output layer constrains non-negativity. HAR-RV-X’s 30-day RV_month regressor’s longer averaging window handles the same distribution shift far better.
- The live-test window is fixed at the moment this notebook is run, not continuously accumulating — re-running it later will naturally score more of the already-elapsed-but-not-yet-scoreable predictions near the end of the current window.

12.2 Future Improvements

- News/sentiment features via an LLM-scored headline pipeline.
- A proper periodic walk-forward refit for GARCH, EWMA, and Holt-Winters, rather than parameters frozen at their training-time values.
- Extending live testing to a longer, continuously-accumulating window.
- A full production migration to 5-minute resolution, rather than the scoped companion pipeline in Section 10 — including GARCH (accepting its far higher computational cost) and the technical-indicator re-scaling a complete switch would require.
- On-chain metrics (exchange netflows, active addresses) as a genuinely independent information source, distinct from every price-history-derived feature tested so far.

References

- Bollerslev, T. (1986). *Generalized Autoregressive Conditional Heteroskedasticity*. Journal of Econometrics, 31(3).
- Corsi, F. (2009). *A Simple Approximate Long-Memory Model of Realized Volatility*. Journal of Financial Econometrics, 7(2).
- Engle, R. (1982). *Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation*. Econometrica, 50(4).
- Glosten, L., Jagannathan, R., & Runkle, D. (1993). *On the Relation between the Expected Value and the Volatility of the Nominal Excess Return on Stocks*. Journal of Finance, 48(5).
- Holt, C. (1957) and Winters, P. (1960). Exponential smoothing with trend and seasonality.
- J.P. Morgan/Reuters (1996). *RiskMetrics — Technical Document* (4th ed.). — the EWMA volatility methodology and its $\lambda=0.94$ industry-standard decay factor.
- Nelson, D. (1991). *Conditional Heteroskedasticity in Asset Returns: A New Approach*. Econometrica, 59(2).

Appendix: Presentation Slides for Sections 11.1–11.2

The two worked examples in Section 11 were also built as standalone presentation slides, reproduced here exactly as delivered (`presentation/capstone-btc-volatility.pptx`, slides 15–16).

Practical Application: Sizing a Trade

Turning this week's real forecast into a position size and a risk estimate



1

Forecast the move

On 2026-08-25, the model predicted BTC would swing about 6% over the coming week — its real, live forecast, not a hypothetical.

2

Size the position

Risking \$50,000 on a move of that size points to a position worth about \$823,000 — bigger forecasted moves mean smaller positions.

3

Estimate the downside

That position carries roughly a 5% chance of losing more than about \$82,000 over the week — the number that goes in a risk report.

4

Compare to the options market

If options are pricing in far more movement than this, they may be overpriced relative to what the model expects — worth a closer look.

Why this is one input, not the final answer

The model explains about 18% of real volatility swings, with typical errors around a quarter of the forecast — useful, but no substitute for market context, judgement, and other risk signals. A more traditional model (GARCH) performed far worse over this same horizon, so this is a genuine improvement, not a finished system.

BTC Volatility Prediction | Capstone

16

Presentation slide accompanying Section 11.1 — Worked Example: Position Sizing and Value-at-Risk.

Practical Application: Pricing an Option

Comparing the model's forecast to how the options market is pricing risk



1

Convert to a yearly figure

The model's short-term forecast, scaled to the yearly figure options pricing uses, comes out to about 44% annualized volatility.

2

Price a hypothetical option

A 7-day BTC option priced at that 44% figure is worth about \$1.177 — the model's own fair-value estimate.

3

Compare to market pricing

If the market prices that same option using 50% volatility instead, it comes out near \$1.454 — about \$280 more expensive than the model suggests is fair.

4

Spot the opportunity

Pricier than the model expects may be worth selling; cheaper (e.g. a 35% assumption, ~\$792) may be worth buying — always alongside the market's own price, never instead of it.

Why this is a signal, not a system

This never overrides a liquid market's own quoted price. It's most useful when no market price exists at all, or for spotting relative mispricing — and even then, it would be sized carefully and combined with other information, not traded on alone.

Presentation slide accompanying Section 11.2 — Worked Example: Options Pricing and Volatility Trading Ideas.